# ODOO

System Administrator
Configuration Guide

Version 17 | April 2026 | Internal Reference

## What this guide covers

This document provides a complete reference for Odoo system administrators — covering initial server setup, database management, user & access control, technical settings, security hardening, performance tuning, backup strategies, and day-to-day maintenance tasks.

## Chapter Overview


| | | | |
|---|---|---|---|
| **1** | Server Installation & Setup | **6** | Security Hardening |
| **2** | Database Configuration | **7** | Backup & Recovery |
| **3** | User & Access Management | **8** | Performance Tuning |
| **4** | Technical Settings | **9** | Module Management |
| **5** | Email & Outgoing Mail | **10** | Monitoring & Logs |

Chapter 1

# Server Installation & Setup

## 1.1 System Requirements

Before installing Odoo, ensure your server meets the following minimum and recommended specifications:

| Component | Minimum | Recommended |
|---|---|---|
| OS | Ubuntu 20.04 LTS | Ubuntu 22.04 / Debian 12 |
| CPU | 2 cores | 4+ cores |
| RAM | 4 GB | 8–16 GB |
| Disk | 40 GB SSD | 100 GB SSD |
| Python | 3.10+ | 3.11 |
| PostgreSQL | 12 | 15 |
| wkhtmltopdf | 0.12.5 | 0.12.6 |

## 1.2 Installation via Package (Recommended)

Install Odoo from the official nightly repository for the most stable setup:

```
# Add Odoo GPG key
wget -O - https://nightly.odoo.com/odoo.key | gpg --dearmor -o
/usr/share/keyrings/odoo-archive-keyring.gpg

# Add repository
echo "deb [signed-by=/usr/share/keyrings/odoo-archive-keyring.gpg]
https://nightly.odoo.com/17.0/nightly/deb/ ./" | tee
/etc/apt/sources.list.d/odoo.list

# Install
apt-get update && apt-get install -y odoo
```

■ **Note:** Always verify the GPG fingerprint from the official Odoo documentation before adding the key.

## 1.3 odoo.conf — Main Configuration File

The primary configuration file is located at **/etc/odoo/odoo.conf**. Below is a reference configuration:

```
[options]
addons_path = /usr/lib/python3/dist-packages/odoo/addons,/opt/odoo/custom-addons
admin_passwd = $pbkdf2-sha512$...(master password hash)...
db_host = localhost
db_port = 5432
db_user = odoo
db_password = StrongDBPassword
db_name = False
db_filter = .*
xmlrpc_port = 8069
longpolling_port = 8072
workers = 4
max_cron_threads = 2
limit_memory_hard = 2684354560
limit_memory_soft = 2147483648
limit_request = 8192
limit_time_cpu = 60
limit_time_real = 120
logfile = /var/log/odoo/odoo.log
log_level = warn
```

■ **Tip:** Use **db_filter** with a regex to restrict which databases are shown per hostname, e.g. *^%d$* limits to the subdomain only.

Chapter 2

# Database Configuration

## 2.1 PostgreSQL Setup

Odoo requires a dedicated PostgreSQL role. Follow these steps to create one:

```
sudo -u postgres createuser -s odoo
sudo -u postgres psql -c "ALTER USER odoo PASSWORD 'StrongDBPassword';"
# Verify
sudo -u postgres psql -c '\du'
```

## 2.2 Database Manager

Access the built-in database manager at **http://<server>:8069/web/database/manager**. From here you can:

- Create a new database (choose language, timezone, demo data)
- Duplicate an existing database for staging/testing
- Delete a database (requires master password)
- Restore a database from a .zip or .dump backup

**■ Warning:** Always set a strong **Master Password** (admin_passwd) before exposing the database manager to a network. Set *list_db = False* in production to hide the list.

## 2.3 Key Database Parameters

| Parameter | Description | Typical Value |
|---|---|---|
| db_host | PostgreSQL host | localhost |
| db_port | PostgreSQL port | 5432 |
| db_user | DB role name | odoo |
| db_password | DB role password | (strong secret) |
| db_name | Default database, False = all | False |
| db_filter | Regex to filter visible DBs | .* |

| Parameter | Description | Typical Value |
| --- | --- | --- |
| db_sslmode | SSL connection mode | prefer |

Chapter 3

# User & Access Management

## 3.1 User Roles

Odoo uses a role-based access model with three primary portal types:

| Role | Description | Cost |
|---|---|---|
| Internal User | Full access to backend modules per security groups | Paid seat |
| Portal User | Access to customer portal only (orders, invoices) | Free |
| Public User | Anonymous website visitors | Free |

## 3.2 Access Rights & Record Rules

Navigate to **Settings** → **Users & Companies** → **Groups** to manage security groups. Each group can have:

- **Access Rights** — Model-level CRUD permissions
- **Record Rules** — Domain-filter rules limiting visible records
- **Implied Groups** — Automatically include permissions from another group
- **Menu Access** — Control which menus are visible

## 3.3 Password Policy

Enable password policies at **Settings** → **Permissions** → **Password Policy**:

| Setting | Recommended Value |
|---|---|
| Minimum length | 12 characters |
| Uppercase letters | Required |
| Digits | Required |

| Setting | Recommended Value |
| --- | --- |
| Special characters | Required |
| Password expiry | 90 days |

■ **Tip:** Enable **Two-Factor Authentication (2FA)** for all internal users from Settings → Integrations → Two-Factor Authentication.

## 3.4 Multi-Company Setup

To enable multi-company: **Settings** → **Users & Companies** → **Companies** → **Enable**. Key considerations:

- Each company has independent chart of accounts, fiscal year, and currency
- Users can be assigned to multiple companies — set the default in their profile
- Inter-company rules (auto purchase orders / invoices) are configured under each company record
- Branches (Odoo 17+) allow shared chart of accounts with location segregation

Chapter 4

# Technical Settings

## 4.1 Developer Mode

Enable developer mode to access technical menus and debugging tools:

- Via URL: append **?debug=1** or **?debug=assets** to any URL
- Via Settings: Settings → scroll to bottom → Activate the Developer Mode
- Via config: set **dev_mode = True** in odoo.conf (server-wide)

**■ Warning:** Never leave developer mode enabled on production for regular users — it exposes internal model structure and bypasses some field validations.

## 4.2 Scheduled Actions (Cron)

Manage automated jobs at **Settings** → **Technical** → **Automation** → **Scheduled Actions**:

| Action | Default Schedule | Notes |
|---|---|---|
| Send Scheduled Emails | Every 1 minute | Controls email queue flushing |
| Auto-post Journal Entries | Daily | Locks previous fiscal periods |
| Session GC | Every 1 hour | Cleans expired HTTP sessions |
| Synchronize Activities | Every day | Reminder notifications |
| Update Currency Rates | Once a day | Requires provider API key |

## 4.3 System Parameters

Low-level key/value configuration at **Settings** → **Technical** → **Parameters** → **System Parameters**:

| Key | Purpose |
|---|---|
| web.base.url | Public base URL used in email links and QR codes |
| auth_password_policy.minlength | Enforce minimum password length |

| Key | Purpose |
| --- | --- |
| mail.catchall.domain | Catch-all inbound email domain |
| database.expiration_date | Enterprise licence expiry date |

Chapter 5

# Email & Outgoing Mail

## 5.1 Outgoing Mail Server

Configure SMTP at **Settings** → **Technical** → **Email** → **Outgoing Mail Servers**:

| Field | Description | Example |
|---|---|---|
| Description | Friendly name | Company SMTP |
| SMTP Server | Host address | smtp.gmail.com |
| SMTP Port | 465 (SSL) or 587 (TLS) | 587 |
| Connection Security | SSL/TLS or STARTTLS | STARTTLS |
| Username | Auth user | user@company.com |
| Password | App password / secret | (app-specific password) |

■ **Tip:** For Gmail, generate an **App Password** (Google Account → Security → 2-Step → App passwords) rather than using your main password.

## 5.2 Incoming Mail (Fetchmail)

Set up incoming mail at **Settings** → **Technical** → **Email** → **Incoming Mail Servers**. Odoo polls the mailbox and routes emails to the correct model (leads, help tickets, etc.) based on the configured alias domain.

- Set the **Alias Domain** under general company settings first
- Each model (CRM Lead, Help Desk Ticket) has its own email alias
- Default polling interval: **5 minutes** via the scheduled action

Chapter 6

# Security Hardening

## 6.1 Master Password

Always set a strong master (admin) password in odoo.conf. Generate a secure hash:

```
python3 -c "from odoo.tools import config;
print(config._check_master_pwd('YourPassword'))"
```

> ■ **Warning:** The master password protects the database manager. If left as the default, anyone can drop or duplicate your databases.

## 6.2 Disable Database Manager in Production

Add these lines to **odoo.conf**:

```
list_db = False
dbfilter = ^productiondb$
```

## 6.3 Firewall & Reverse Proxy

Only ports 80/443 should be publicly accessible. Odoo should listen on localhost only and be proxied via Nginx or Apache:

```
# /etc/nginx/sites-available/odoo
upstream odoo { server 127.0.0.1:8069; }
upstream odoochat { server 127.0.0.1:8072; }

server {
listen 443 ssl http2;
server_name odoo.example.com;
ssl_certificate /etc/ssl/certs/odoo.crt;
ssl_certificate_key /etc/ssl/private/odoo.key;

proxy_read_timeout 720s;
location / { proxy_pass http://odoo; }
location /longpolling { proxy_pass http://odoochat; }
}
```

■ **Note:** Add **proxy_mode = True** in odoo.conf when running behind a reverse proxy so that Odoo trusts the X-Forwarded-For header.

Chapter 7

# Backup & Recovery

## 7.1 Built-in Backup via UI

Navigate to **http://<server>:8069/web/database/manager**, click **Backup** next to the database. Choose:

- **ZIP** — Includes database dump + filestore (all attachments). Portable, larger size.
- **pg_dump** — Database only. Faster, requires separate filestore backup.

## 7.2 Automated Backup Script

```
#!/bin/bash
BACKUP_DIR=/var/backups/odoo
DB_NAME=productiondb
DATE=$(date +%Y%m%d_%H%M%S)
mkdir -p $BACKUP_DIR

# Database dump
pg_dump -U odoo $DB_NAME | gzip > $BACKUP_DIR/${DB_NAME}_${DATE}.sql.gz

# Filestore
tar -czf $BACKUP_DIR/filestore_${DATE}.tar.gz
/var/lib/odoo/.local/share/Odoo/filestore/$DB_NAME/

# Remove backups older than 30 days
find $BACKUP_DIR -type f -mtime +30 -delete
echo 'Backup completed: '$DATE
```

■ **Tip:** Schedule this script with **cron** (e.g. daily at 02:00 AM) and copy backups offsite to S3 / SFTP / cloud storage.

## 7.3 Restore Procedure

**Step 1.** Create a fresh empty database: *createdb -U postgres newdb*

**Step 2.** Restore SQL dump: *gunzip -c backup.sql.gz | psql -U odoo newdb*

**Step 3.** Restore filestore: *tar -xzf filestore.tar.gz -C /var/lib/odoo/.local/share/Odoo/filestore/newdb/*

**Step 4.** Start Odoo pointing at the restored database and verify data integrity

Chapter 8

# Performance Tuning

## 8.1 Worker Configuration

For production multi-user deployments, use multi-process (worker) mode. Recommended formula:

```
workers = (2 × CPU_cores) + 1 # e.g. 4 cores → 9 workers
max_cron_threads = 2 # keep low to reserve CPU for requests
```

| Parameter | Description | Default |
|---|---|---|
| workers | HTTP worker processes (0 = threaded mode) | 0 |
| max_cron_threads | Cron worker processes | 2 |
| limit_memory_hard | Hard memory limit per worker (bytes) — kills worker | 2.5 GB |
| limit_memory_soft | Soft memory limit — graceful recycle | 2 GB |
| limit_time_cpu | Max CPU time per request (s) | 60 |
| limit_time_real | Max wall-clock time per request (s) | 120 |
| limit_request | Max requests per worker before recycle | 8192 |

## 8.2 PostgreSQL Tuning

Key postgresql.conf settings for an Odoo workload (16 GB RAM server example):

```
shared_buffers = 4GB # 25% of RAM
work_mem = 32MB # per sort/join operation
maintenance_work_mem = 1GB # for VACUUM, CREATE INDEX
effective_cache_size = 12GB # 75% of RAM
max_connections = 200 # must cover workers + cron
checkpoint_completion_target = 0.9
wal_buffers = 64MB
```

```
random_page_cost = 1.1 # for SSD storage
```

■ **Tip:** Use **pgBadger** to analyse PostgreSQL slow query logs and identify missing indexes. Run *EXPLAIN ANALYSE* on slow Odoo reports.

Chapter 9

# Module Management

## 9.1 Installing & Updating Modules

- Via UI: Apps menu → search → Install / Upgrade
- Via CLI (recommended for bulk operations):

```
# Install a module
odoo -c /etc/odoo/odoo.conf -d mydb --stop-after-init -i sale_management

# Update a module
odoo -c /etc/odoo/odoo.conf -d mydb --stop-after-init -u sale_management

# Update ALL installed modules (use with caution)
odoo -c /etc/odoo/odoo.conf -d mydb --stop-after-init -u all
```

■ **Warning:** Always take a database backup before running **-u all** or major module upgrades. Test in a staging environment first.

## 9.2 Custom Add-ons Path

Place custom modules in a dedicated directory and add it to **addons_path**:

```
addons_path = /usr/lib/python3/dist-packages/odoo/addons,
/opt/odoo/extra-addons,
/opt/odoo/custom-addons
```

■ **Note:** Each directory in addons_path must contain at least one module folder with a valid **__manifest__.py** file.

Chapter 10

# Monitoring & Logs

## 10.1 Log Configuration

| Level | Use Case |
|---|---|
| critical | Only fatal errors — minimal logging for high-traffic production |
| error | Errors only |
| warn | Recommended for production |
| info | General operational information |
| debug | Verbose — use only for troubleshooting, never in production |

```
logfile = /var/log/odoo/odoo.log
log_level = warn
logrotate = True # built-in log rotation
```

## 10.2 Useful Log Commands

```
# Tail live log
tail -f /var/log/odoo/odoo.log

# Filter for errors only
grep -i 'error\|traceback\|critical' /var/log/odoo/odoo.log | less

# Check Odoo service status
systemctl status odoo
journalctl -u odoo -n 100 --no-pager
```

## 10.3 Health Check Endpoint

Odoo exposes a simple health check URL for load balancers and uptime monitors:

```
curl -I http://localhost:8069/web/health
```

Returns HTTP 200 when the server is ready. Use this as the health check URL in HAProxy, Nginx upstream checks, or monitoring tools like Zabbix / Prometheus Blackbox Exporter.

■ **Tip:** Monitor the **longpolling port (8072)** separately — a healthy main port does not guarantee the chat / live-chat worker is running.